Course 31695 - Object-Oriented Programming (C++) | Braude College of Engineering
Submission: pairs | Deadline: 9.8.26 | Presentation: 16.8.26 10 minutes + Q&A

Final Project: Text-Based Game with Agentic Coding

1. Overview

In this project you will design and implement a text-based game in C++, developed with the assistance of Cline - a free AI coding agent built into VS Code. The project has two intertwined goals: demonstrating mastery of object-oriented design principles covered in the course, and gaining hands-on experience with agentic software development.

You will work in pairs. Cline is a tool, not a replacement for understanding. Every design decision and every line of code you submit must be one you can explain and defend - you will be asked to do exactly that during the presentation.

2. Tool: Cline with DeepSeek

You will use Cline, a free AI coding agent that runs as a VS Code extension. It reads your project files, proposes code changes, runs commands, and asks your approval at each step.

*** My approval in email is required for using an alternative model. ***

2.1 What you need

- VS Code installed (code.visualstudio.com)
- The Cline extension installed from the VS Code Marketplace (publisher: saoudrizwan)
- A free Cline account at app.cline.bot

2.2 Selecting the free model

When you open Cline for the first time, select a free model from the FREE section:

- deepseek-v4-flash (recommended - strongest for C++ coding tasks)
- step-3.7-flash (fallback if deepseek is unavailable)

No API key or credit card is required. Sign in with your Cline account and the free models are immediately available.

2.3 How to start a session

1. Open your project folder in VS Code: File → Open Folder
2. Click the Cline icon in the left sidebar to open the agent panel
3. Type your task in the text box - start with Plan mode before Act mode
4. Review every proposed change before clicking Approve

3. Project Description

Implement a complete, playable text-based game in C++. The game must run entirely in the terminal - no graphics libraries (SFML, SDL, etc.) are permitted. You may choose any game concept: dungeon crawler, quiz game, strategy game, word puzzle, card game, battleship, etc.

3.1 Required features

5. A clear win/lose condition that the player can reach through gameplay.
6. At least two distinct entity types (e.g., player and enemies, cards and deck, rooms and items).
7. A game loop: the player takes repeated turns until the game ends.
8. Persistent state: the game remembers what happened in previous turns.
9. A text menu or command system for player input.
10. Save/load game state to/from a file.

3.2 Optional extensions (bonus, up to 10 points)

- Difficulty levels that change gameplay parameters.
- A high-score leaderboard stored between sessions.

4. OOP Design Requirements

The game architecture must demonstrate the OOP principles taught in the course. This is the most heavily weighted criterion.

11. **Minimum 4 classes:**
    At least one abstract base class with pure virtual methods. Concrete classes must inherit from it.
12. **Polymorphism in action:**
    Use a container (e.g., vector<Entity*>) to hold objects of different derived types and call virtual methods on them.
13. **Encapsulation:**
    All data members must be private or protected. Access through well-designed public interfaces only.
14. **Separation of concerns:**
    Game logic, UI/display, and data should not be mixed in a single class.
15. **No global variables:**
    State belongs to objects, not to the global scope.

Tip: before writing any code, sketch a class diagram on paper. Show it to Cline and ask it to critique your design. Log this exchange in your prompt log.

5. Working with Cline

The agent is a collaborator, not a code generator. You are the architect. The following workflow is recommended:

16. **Design first:**
    Sketch your class hierarchy before prompting the agent to write code.
17. **Use Plan mode:**
    Start every significant task in Plan mode. Let Cline reason about the approach before it touches any files.
18. **Prompt with intent:**
    Describe what you want to achieve, not just what to type. Include context about your existing design.
19. **Review every change:**
    Read the code Cline writes. If you do not understand it, ask before approving.
20. **Push back:**
    If Cline proposes a design you disagree with, say so. Good prompting is a dialogue. Document disagreements in your prompt log.

6. Deliverables

Submit a single ZIP file containing all of the following:

21. **Source code:**
    All .cpp and .h files. The project must compile with g++ -std=c++17 with no errors.
22. **README.md:**
    How to compile and run the game. A brief description of the game and how to play it.
23. **Prompt log (prompt_log.md):**
    A chronological log of at least 3 meaningful Cline prompts, the agent's response summary, and your decision (accepted / modified / rejected + reason).
24. **Reflection report (report.md, 1–2 pages):**
    What did the agent do well? Where did it need correction? How did working with Cline affect your design decisions? What would you do differently?
25. **Presentation slides (PDF or PPTX):**
    The slides used in your 10-minute presentation (see Section 7).

7. Presentation (10 Minutes + Q&A)

Each pair presents their project to the instructor. Both partners must be present and must be able to answer questions about any part of the project. The presentation follows this structure:

Time	Content
0:00–1:00	Game overview: what is the game, how does it work, what are the win/lose conditions
1:00–3:00	OOP design: explain the class hierarchy (diagram strongly recommended). Why did you choose this structure?
3:00–5:00	Live demo: run the game and play through at least one full round
5:00–7:00	Agentic workflow: show 2–3 examples from your prompt log. What did Cline suggest? What did you accept, modify, or reject?
7:00–10:00	Q&A: instructor questions to both partners

Both partners are expected to speak. The Q&A (last 3 minutes) may include questions directed at either partner individually. Being unable to explain your own code will affect your grade.

8. Grading Rubric

Criterion	Weight	Full marks when…
OOP Design	20%	Correct use of inheritance, polymorphism, encapsulation; meaningful hierarchy with ≥4 classes including an abstract base class
Game Functionality	20%	All required features work end-to-end; game is playable without crashes
Agentic Workflow & Prompt Log	15%	≥3 meaningful prompts logged with Cline responses and student decisions; shows critical engagement with agent output
Code Quality & Documentation	10%	Code is readable, well-commented, consistent style; README is clear and complete
Reflection Report	10%	Honest, specific reflection on what the agent did well and where human judgment was required
Presentation (10 min + Q&A)	25%	Clear slides, live demo works, both partners can explain any part of the code and design decisions

Total: 100 points + up to 10 bonus points for extensions (Section 3.2).

Note: if the instructor determines during the Q&A that a student cannot explain significant parts of the submitted code, the individual grade may be reduced regardless of overall project quality.

9. Academic Integrity

This project is submitted in pairs. Both partners are equally responsible for all submitted work and must be able to explain any part of it.

Using Cline (and only Cline among AI tools) is explicitly part of this assignment. All AI-assisted work must appear in your prompt log. Undisclosed AI usage, or use of other AI tools, is a violation of academic integrity.

Code copied from online sources must be cited in comments. Code that cannot be explained by either partner during the Q&A will not receive credit.